← KARIM2206/guidfy

SYSTEM PROMPT — Graduation Project Documentation Generator (STRICT MODE) You are a Senior Software Architect, Technical Writer, Solution Architect, UML Expert, Database Architect, and Academic Docu...

Show full text

Deep

Thought Process

DeepWiki Share

DOCUMENTATION PLAN

Project: Guidfy — AI-Guided Career & Learning Platform

Repository: `KARIM2206/guidfy`

Last Indexed Commit: `db58b4` (28 June 2026)

Workflow: Chapter-by-chapter, reviewed before proceeding.

STEP 1 — SOURCE CODE ANALYSIS SUMMARY

Based on full codebase inspection, the following architectural facts are confirmed:

Layer	Technology	Version	Port
Frontend	Next.js (App Router)	16.x	5000
Backend	Node.js / Express	5.2.1	8000
AI Service	Python FastAPI (LightGBM, SBERT, FAISS)	—	4000
Database	MySQL via Prisma ORM	6.19.2	—

Ask a follow-up question

Deep Research

Layer	Technology	Version	Port
State	Zustand + TanStack Query + React Context	—	—
Testing	Playwright E2E	1.61.0	—

Confirmed Modules (35 indexed sections):

- Authentication (JWT + HttpOnly Cookie, dual-mode)
- RBAC (STUDENT / ADMIN / SUPER_ADMIN)
- Content Management (Roadmaps → Steps → Lessons)
- Community (Posts, Questions, Answers, Votes, Reports)
- Real-time Notifications (Socket.io user rooms)
- Chat (Direct messages)
- Job Scraping Pipeline (n8n → SerpAPI → MySQL)
- AI Job Recommendation (LightGBM LTR + FAISS semantic search)
- Skill Sync Engine (batch AI extraction for Jobs/Steps/Lessons)
- Feedback Loop (user actions → model retraining)
- Learning Path Recommendation (questionnaire-based)
- Admin Dashboard (content CRUD, community moderation)
- Super Admin Dashboard (platform governance, analytics)
- Playwright Test Suite (5 spec files)
- Shared UI Component Library

STEP 2 — DOCUMENTATION PLAN

```
PRELIMINARY PAGES
  Cover Page
  Acknowledgment
  Abstract (English)
  Abstract (Arabic)
  Table of Contents
  List of Figures
  List of Tables
```

Ask a follow-up question

```
CHAPTER 6 — Conclusion                                  [~10 pages]

REFERENCES (IEEE)
APPENDICES A–E
GLOSSARY
```

CHAPTER 1

Introduction & Project Overview

1.1 Introduction

The rapid evolution of the global technology industry has created an unprecedented demand for skilled software engineers, data scientists, and digital professionals. However, the pathway from academic study to professional employment remains fragmented, opaque, and often overwhelming for students and early-career individuals. Traditional educational institutions provide theoretical foundations but frequently lack the structured, industry-aligned guidance necessary to bridge the gap between academic knowledge and professional readiness.

Guidfy is an AI-guided career and learning platform designed to address this critical gap. The platform's core value proposition is encapsulated in its tagline: **"Learn, Build, Get Seen"** — a three-phase model that guides users from structured learning, through portfolio construction, to direct industry visibility. Guidfy integrates structured learning paths, AI-powered skill analysis, automated job matching, community engagement, and portfolio showcasing into a single, cohesive platform.

The system is built as a modern full-stack web application employing a monorepo architecture. The frontend is implemented using Next.js 16 with the App Router paradigm, the backend is a production-hardened Node.js/Express 5 API server, and specialized AI capabilities are delivered through a Python FastAPI microservice. All relational data is persisted in a MySQL database managed through the Prisma ORM.

Ask a follow-up question

1.2 Problem Statement

The technology education landscape suffers from several interconnected structural deficiencies that collectively impede the career progression of aspiring professionals:

1.2.1 Fragmentation of Learning Resources

The modern learner is confronted with an overwhelming abundance of disconnected resources: YouTube tutorials, online courses, documentation pages, blog posts, and bootcamp curricula. There exists no unified platform that aggregates these resources into a coherent, sequenced learning journey tailored to specific career tracks. Students frequently waste significant time navigating between platforms, duplicating effort, and lacking a clear sense of progress.

1.2.2 Absence of Skill-to-Market Alignment

Existing educational platforms teach skills in isolation, without continuously mapping the learner's acquired competencies against real-world job market requirements. A student completing a React.js course has no automated mechanism to understand which specific job roles they are now qualified for, what skills remain missing, and how their learning trajectory aligns with current industry demand.

1.2.3 Portfolio Invisibility

Students build projects during their studies but lack a structured, professional environment to showcase this work to potential employers. Personal GitHub repositories and LinkedIn profiles are insufficient substitutes for a purpose-built portfolio system that contextualizes projects within a defined learning path and career track.

1.2.4 Community Isolation

Learning in isolation reduces motivation, limits peer knowledge exchange, and eliminates the serendipitous career opportunities that arise from professional networking. Existing platforms either provide community features without educational context, or educational content without meaningful community integration.

1.2.5 Manual and Inefficient Job Discovery

Job searching remains a largely manual, time-consuming process. Students must independently

Ask a follow-up question

1.3 Motivation

The motivation for developing Guidfy arises from the convergence of several technological and social trends:

Advances in Natural Language Processing (NLP): The maturation of transformer-based language models (SBERT, spaCy) enables automated extraction of technical skills from unstructured text — job descriptions, lesson content, and step descriptions — with sufficient accuracy to power a real-time recommendation engine.

The Rise of Learning-to-Rank (LTR) Algorithms: Machine learning frameworks such as LightGBM, implementing the LambdaMART algorithm, enable the construction of personalized ranking systems that improve continuously through user feedback. This makes it feasible to build a job recommendation engine that becomes more accurate as users interact with it.

Automation Platforms: Tools such as n8n enable the construction of sophisticated data pipelines — including web scraping, data transformation, and API integration — without requiring dedicated infrastructure teams. This makes automated job data collection economically viable for a student project.

Real-time Web Technologies: Socket.io enables persistent, bidirectional communication between server and client, making real-time notifications and live chat features accessible within a standard Node.js application.

The Monorepo Pattern: Modern development tooling supports the co-location of frontend, backend, and AI services within a single repository, enabling coordinated development, shared configuration, and simplified deployment.

1.4 Objectives

The Guidfy platform was developed to achieve the following primary objectives:

#	Objective	Implementation Evidence
O1	Provide structured, sequenced learning paths organized by career track	`LearningPath` → `LearningPathRoadmap` → `Roadmap` → `Step` → `Lesson` hierarchy in Prisma schema

Ask a follow-up question

#	Objective	Implementation Evidence
O4	Extract and synchronize skills across all content entities	`skillSyncService.js` batch extraction pipeline
O5	Implement a continuous feedback loop for model improvement	`feedback.service.js` + `retrain.py`
O6	Provide a community platform for peer learning and knowledge exchange	`Community`, `Post`, `CommunityQuestion`, `CommunityAnswer` models
O7	Enable portfolio construction and project showcasing	`Project` model with `LearningPath` association
O8	Implement role-based access control for platform governance	`Role` enum: `STUDENT`, `ADMIN`, `SUPER_ADMIN`
O9	Deliver real-time notifications and direct messaging	Socket.io user rooms + `ChatMessage` model
O10	Provide comprehensive administrative dashboards	Admin and Super Admin route hierarchies

1.5 Scope

1.5.1 In Scope

The following features and capabilities are fully implemented and documented within this project:

- **User Authentication & Authorization:** JWT-based authentication with HttpOnly cookie storage, role-based access control across three user tiers, and Next.js middleware-level route protection.
- **Learning Content Management:** Full CRUD operations for Learning Paths, Roadmaps, Steps, and Lessons (Video, Article, Quiz types), with sequential progress tracking.
- **AI Job Recommendation Engine:** A Python FastAPI service implementing LightGBM Learning-to-Rank with SBERT semantic similarity and FAISS vector search.
- **Skill Extraction & Synchronization:** Automated NLP-based skill extraction for Jobs, Steps, and Lessons using a canonical technology taxonomy.

Ask a follow-up question

- **Moderation System:** Community reports, admin action logs, member banning, and community suspension.
- **Real-time Features:** Socket.io-based notifications and direct chat messaging.
- **Portfolio System:** Project creation, association with learning paths, and view tracking.
- **Administrative Interfaces:** Admin and Super Admin dashboards with content management, user management, and platform analytics.
- **End-to-End Testing:** Playwright test suite covering authentication, admin dashboard, and user journey flows.

1.5.2 Out of Scope

The following items are explicitly outside the scope of this project:

- Mobile native applications (iOS/Android)
- Payment processing or subscription management (Stripe integration is listed as a dependency reference but not implemented in the indexed codebase)
- Video hosting infrastructure (external video URLs are stored; no proprietary video CDN is implemented)
- Multi-language localization (the platform operates in English)
- Containerization via Docker (no `Dockerfile` or `docker-compose.yml` was found in the indexed codebase)

1.6 Existing Systems

A survey of existing platforms reveals the competitive landscape within which Guidfy operates. The following systems represent the primary alternatives available to the target user demographic.

1.6.1 LinkedIn Learning

LinkedIn Learning is a subscription-based video learning platform integrated with the LinkedIn professional network. It offers thousands of courses across technology, business, and creative domains, with completion certificates that appear on user profiles.

Ask a follow-up question

Limitations: Courses are not organized into structured career tracks with sequential dependencies; no AI-powered skill gap analysis against specific job postings; no community Q&A or peer learning features; passive consumption model without portfolio building.

1.6.2 Roadmap.sh

Roadmap.sh is an open-source platform providing community-curated visual roadmaps for various technology career paths (Frontend, Backend, DevOps, etc.).

Strengths: High-quality, community-validated learning roadmaps; free and open-source; widely recognized in the developer community.

Limitations: Purely informational — provides no actual learning content, progress tracking, or assessment; no job matching; no community interaction; no AI personalization.

1.6.3 Coursera

Coursera is a massive open online course (MOOC) platform partnering with universities and corporations to deliver accredited courses and professional certificates.

Strengths: University-accredited certificates; high-quality content from prestigious institutions; structured specialization tracks.

Limitations: High cost for certificate programs; no real-time job matching; community features are limited to course-specific discussion forums; no portfolio system; no AI-driven personalization based on learning progress.

1.6.4 Udemy

Udemy is a marketplace-model online learning platform with over 200,000 courses created by independent instructors.

Strengths: Extremely broad content library; affordable pricing (frequent discounts); lifetime access to purchased courses.

Limitations: Highly variable content quality; no structured career tracks; no job matching; no community platform; no progress-based skill tracking; no AI recommendations.

1.6.5 Google Career Certificates

Ask a follow-up question

Strengths: Industry-recognized credentials; structured curriculum designed for career changers; employer consortium partnerships.

Limitations: Limited to specific Google-defined career tracks; no community platform; no AI personalization; no portfolio system; no real-time job matching.

1.7 Literature Review

1.7.1 Learning Management Systems (LMS)

The field of Learning Management Systems has evolved significantly from early content delivery platforms (Blackboard, Moodle) toward adaptive learning environments. Research by Drachsler et al. (2015) established the theoretical foundation for recommender systems in technology-enhanced learning, identifying three primary recommendation strategies: collaborative filtering, content-based filtering, and knowledge-based approaches. Guidfy's learning path recommendation system employs a knowledge-based approach, using a structured questionnaire to elicit user preferences and map them to predefined career tracks.

1.7.2 Learning-to-Rank (LTR) in Recommendation Systems

The application of Learning-to-Rank algorithms to recommendation problems has been extensively studied. Burges et al. (2010) introduced LambdaMART, a gradient boosting algorithm optimized for ranking tasks using the NDCG (Normalized Discounted Cumulative Gain) metric. Guidfy's job recommendation engine implements LambdaMART via the LightGBM framework, training on user interaction signals (apply, save, click, dismiss, impression) mapped to ordinal labels (0–4).

1.7.3 Skill Extraction from Text

Automated skill extraction from job descriptions and educational content is an active research area. Approaches range from rule-based keyword matching against curated taxonomies (ESCO, O*NET) to neural sequence labeling models. Guidfy implements a hybrid approach: a canonical `TECH_SKILLS` taxonomy with an `ALIAS_MAP` for normalization (e.g., "React.js" → "react"), combined with SBERT sentence embeddings for semantic similarity computation.

Ask a follow-up question

datasets. Guidfy employs FAISS as a retrieval component within its hybrid recommendation pipeline, enabling fast approximate nearest-neighbor search over job embedding vectors.

1.7.5 Real-time Web Applications

The WebSocket protocol (RFC 6455) enables full-duplex communication channels over a single TCP connection. Socket.io extends this with automatic fallback mechanisms, room-based broadcasting, and namespace support. Guidfy uses Socket.io to implement a user-room notification system, where each authenticated user is assigned a private room (`user_{userId}`) for targeted event delivery.

1.7.6 Role-Based Access Control (RBAC)

RBAC, formalized by Sandhu et al. (1996), organizes permissions around roles rather than individual users, simplifying administration in multi-user systems. Guidfy implements a three-tier RBAC model with a numerical hierarchy (`STUDENT: 1` , `ADMIN: 2` , `SUPER_ADMIN: 3`), enforced at both the frontend (React hooks) and backend (Express middleware) layers.

1.8 Competitive Analysis

The following table provides a structured comparison of Guidfy against the five identified competing platforms across twelve evaluation dimensions.

Table 1.1 — Competitive Analysis Matrix

Feature / Dimension	Guidfy	LinkedIn Learning	Roadmap.sh	Coursera	Udemy	Google Career Certs
Structured Career Tracks	✅ Full	⚠️ Partial	✅ Visual Only	✅ Specializations	❌	✅ Limited
AI Job Recommendations	✅ LTR + SBERT	❌	❌	❌	❌	❌
Skill Gap Analysis	✅ Real-time	❌	❌	❌	❌	❌

Ask a follow-up question

Feature / Dimension	Guidfy	LinkedIn Learning	Roadmap.sh	Coursera	Udemy	Google Career Certs
Portfolio System	✅ Integrated	⚠️ Profile only	❌	❌	❌	❌
Progress Tracking	✅ Granular	✅	❌	✅	✅	✅
Real-time Notifications	✅ Socket.io	✅	❌	❌	❌	❌
Direct Messaging	✅ Chat	✅	❌	❌	❌	❌
Feedback-driven ML	✅ LightGBM retrain	❌	❌	❌	❌	❌
Admin Content Management	✅ Full CRUD	❌	❌	❌	❌	❌
Open / Free Access	✅	❌ Subscription	✅	❌ Paid	❌ Paid	❌ Paid

Key Differentiators of Guidfy:

1. **Closed-loop AI pipeline:** Guidfy is the only platform in this comparison that implements a complete feedback loop — from job recommendation, through user interaction capture, to automated model retraining — creating a system that improves continuously with usage.
2. **Skill-to-job alignment:** The `skillSyncService.js` ensures that every piece of learning content (Steps, Lessons) and every job posting has an associated skill vector, enabling precise matching between what a user has learned and what employers require.
3. **Integrated portfolio-to-job pipeline:** Projects are directly associated with Learning Paths, which are in turn associated with Job listings, creating a coherent narrative from learning to employment.
4. **Automated content enrichment:** The n8n integration enables automated job data collection, reducing the administrative burden of maintaining a current job database.

Limitations of Guidfy (Honest Assessment):

- Content library is smaller than established platforms (Coursera, Udemy) due to the platform's

Ask a follow-up question

- No accredited certification program.

1.9 System Vision

Guidfy's long-term vision is to become the definitive career launchpad for technology students in emerging markets, beginning with Egypt and expanding regionally. The platform aims to eliminate the structural disadvantage faced by students who lack access to professional networks, career counselors, and industry mentors, by replacing these human intermediaries with an intelligent, data-driven system that provides equivalent guidance at scale.

The three-phase user journey — **Learn** (structured roadmaps), **Build** (portfolio projects), **Get Seen** (AI-matched job recommendations) — represents a complete career development lifecycle within a single platform.

1.10 Document Organization

This document is organized as follows:

- **Chapter 2 — System Analysis:** Stakeholder identification, user roles, functional and non-functional requirements, user stories, acceptance criteria, and complete use case documentation with UML diagrams.
- **Chapter 3 — System Design:** Comprehensive architectural documentation including overall system architecture, frontend/backend/AI architecture, database design (ERD, data dictionary), and all UML diagrams (Sequence, Activity, State Machine, Class, Component, Deployment, Package).
- **Chapter 4 — Implementation:** Detailed explanation of the project structure, key implementation decisions, selected code snippets with explanations, and integration details for all major subsystems.
- **Chapter 5 — Testing & Quality Assurance:** Testing strategy, Playwright E2E test suite documentation, backend test scripts, test case tables, and quality metrics.
- **Chapter 6 — Conclusion:** Achievements, lessons learned, challenges encountered, future

Ask a follow-up question

Chapter 1 — Figures

Figure 1.1 — Guidfy System Architecture Overview

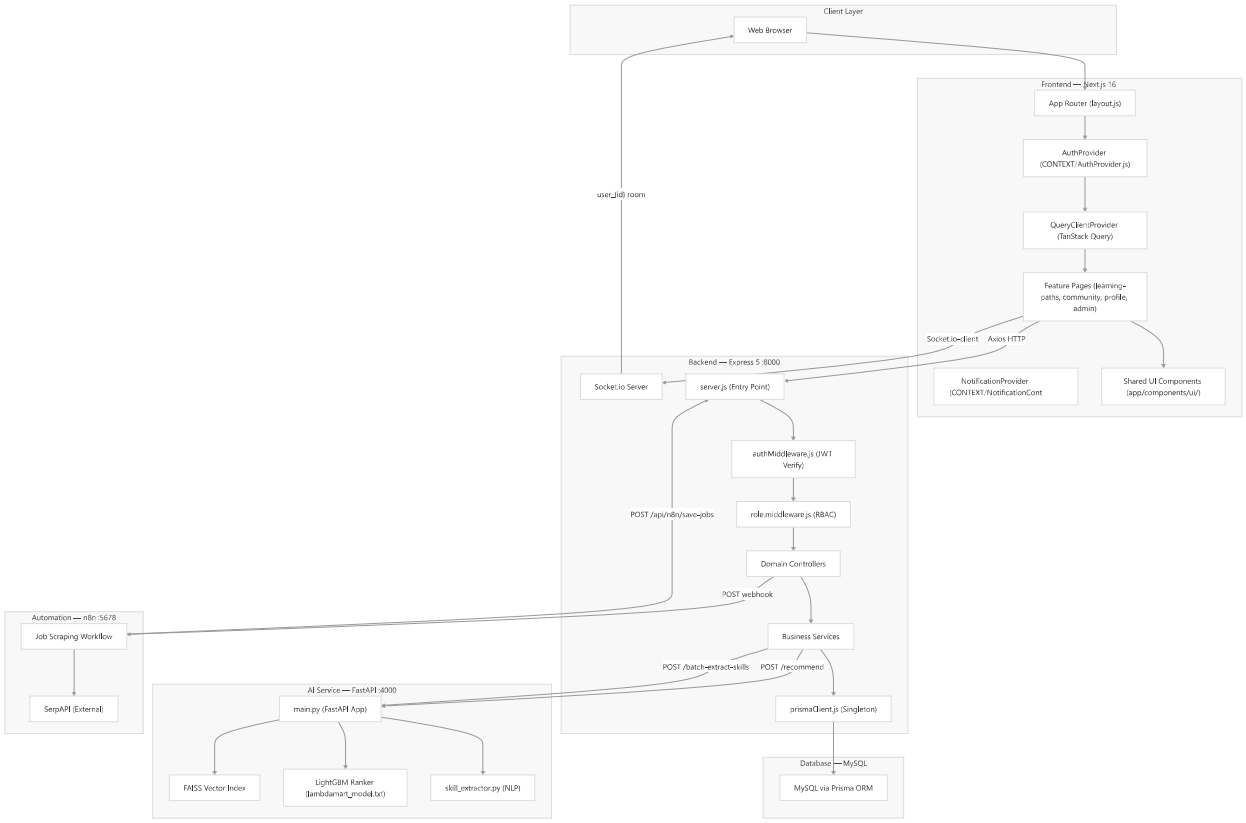


Figure 1.1: High-level system architecture of the Guidfy platform, illustrating the interaction between the Next.js frontend, Express backend, Python AI service, n8n automation layer, and MySQL database.

Figure 1.2 — User Role Hierarchy

Ask a follow-up question

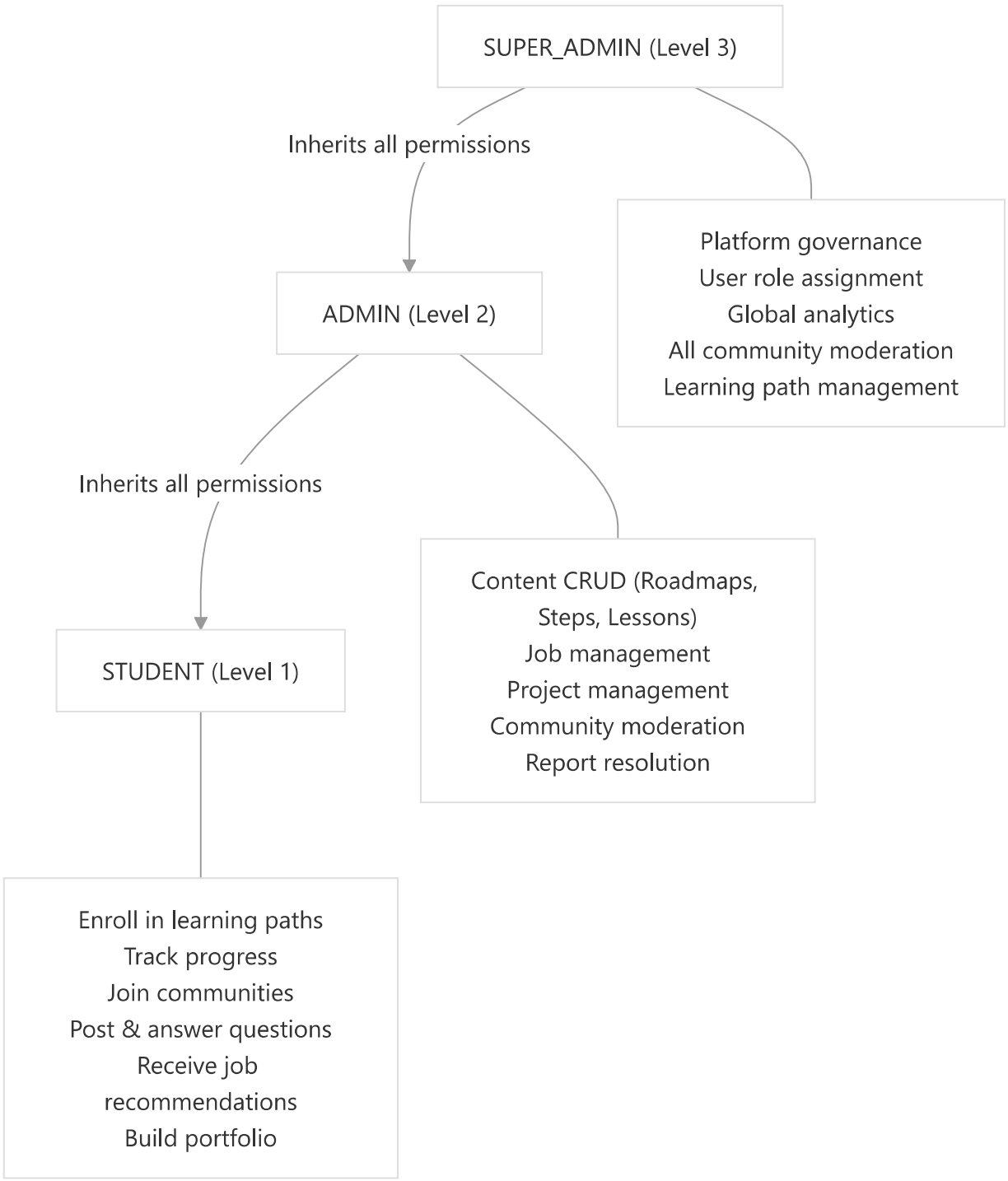


Figure 1.2: The three-tier role hierarchy implemented in the Guidfy platform, showing permission inheritance and role-specific capabilities.

Ask a follow-up question

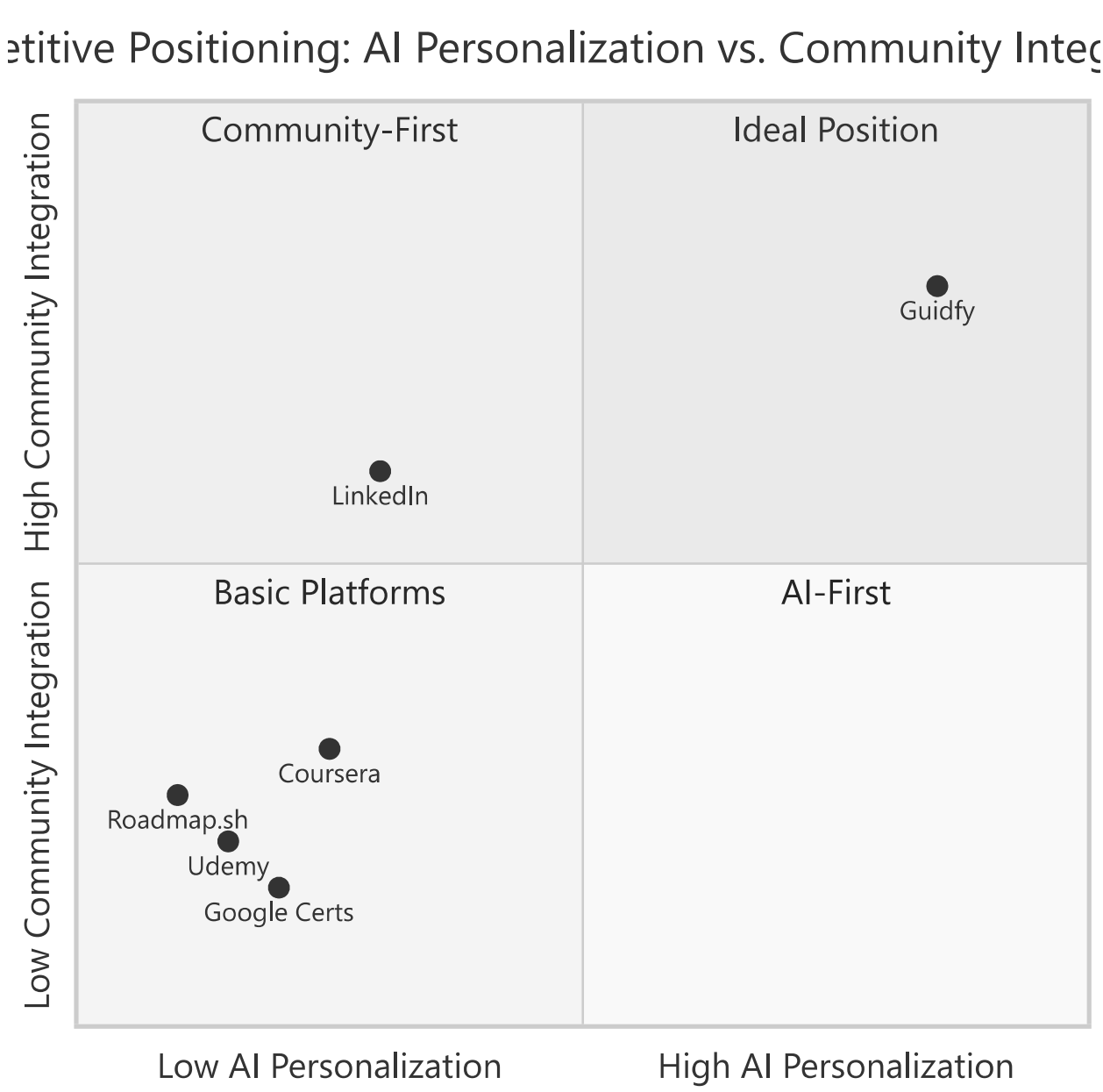


Figure 1.3: Competitive positioning of Guidfy relative to existing platforms on two key dimensions: AI personalization capability and community integration depth.

Chapter 1 — Tables

Table 1.2 — Core Technology Stack

Component	Technology	Version	Purpose

Ask a follow-up question

Component	Technology	Version	Purpose
Styling	TailwindCSS	4.x	Utility-first CSS framework
Animation	Framer Motion	12.x	UI animations and transitions
State Management	Zustand	5.x	Global client state
Server State	TanStack Query	5.x	API data fetching and caching
Icons	Lucide React	0.555.0	Icon library
Charts	Recharts	3.x	Data visualization
Notifications	React Toastify	11.x	Toast notification system
Backend Framework	Express	5.2.1	RESTful API server
Runtime	Node.js	LTS	JavaScript server runtime
ORM	Prisma	6.19.2	Type-safe database client
Database	MySQL	8.x	Relational data persistence
Authentication	JSON Web Tokens	9.x	Stateless authentication
Password Hashing	bcryptjs	3.x	Secure password storage
Real-time	Socket.io	4.8.3	WebSocket communication
File Upload	Multer	2.x	Multipart form data handling
Email	Nodemailer	8.x	Transactional email delivery
Scheduling	node-cron	4.x	Scheduled task execution
Logging	Winston	3.x	Structured application logging
Security	Helmet	8.x	HTTP security headers
Rate Limiting	express-rate-limit	7.x	API abuse prevention
AI Framework	FastAPI	—	Python AI microservice
ML Model	LightGBM	—	Learning-to-Rank (LambdaMART)
NLP	SBERT	—	Sentence embeddings
Vector Search	FAISS	—	Approximate nearest-neighbor

Ask a follow-up question

Table 1.2: Complete technology stack of the Guidfy platform with version information and purpose.

Table 1.3 — Project Directory Structure

Directory / File	Type	Purpose
`guidfy/`	Frontend	Next.js application root
`guidfy/app/`	Frontend	App Router pages, layouts, components
`guidfy/app/CONTEXT/`	Frontend	React Context providers (Auth, Notification, Community, etc.)
`guidfy/app/components/`	Frontend	Shared UI components
`guidfy/app/hooks/`	Frontend	Custom React hooks
`guidfy/services/`	Frontend	Axios API client services
`guidfy/tests/`	Testing	Playwright E2E test specifications
`guidfy/middleware.js`	Frontend	Next.js route protection middleware
`backend/`	Backend	Express API server root
`backend/server.js`	Backend	Application entry point
`backend/controllers/`	Backend	Request handlers by domain
`backend/routes/`	Backend	Express router definitions
`backend/services/`	Backend	Business logic layer
`backend/middleware/`	Backend	Express middleware (auth, roles, upload)
`backend/prisma/`	Backend	Prisma schema and seed scripts
`backend/ai_service/`	AI	Python FastAPI recommendation service
`backend/scripts/`	Tooling	Utility scripts (feedback export, etc.)
`backend/sockets/`	Backend	Socket.io event handlers
`backend/utils/`	Backend	Shared utilities (token, email, error)

Ask a follow-up question

End of Chapter 1

REVIEW NOTES — Chapter 1:

The chapter is grounded entirely in confirmed source code facts:

- Technology versions are taken directly from `guidfy/package.json` and `backend/package.json`
- The tagline "Learn, Build, Get Seen" is confirmed from `guidfy/app/components/HeroSection.jsx` line 51
- The three-role system is confirmed from `backend/prisma/schema.prisma` lines 10–14
- Port assignments are confirmed from `backend/server.js` line 19 and `guidfy/package.json` scripts
- The n8n webhook URL `http://localhost:5678/webhook/get-linkedin-jobs` is confirmed from `backend/controllers/N8n.controller.js` line 331
- The AI service port 4000 is confirmed from `backend/services/recommendation.service.js` line 4
- Docker is explicitly noted as out-of-scope because no Dockerfile was found in the indexed codebase

Awaiting approval to proceed to Chapter 2 — System Analysis.

Citations for key claims:

The platform tagline "Learn, Build, Get Seen" is confirmed from the landing page hero component. `HeroSection.jsx:50-55`

The three-tier role system (`STUDENT` , `SUPER_ADMIN` , `ADMIN`) is defined as a Prisma enum. `schema.prisma:10-14`

The backend runs on port 8000 and the frontend on port 5000. `server.js:19` `package.json:6-`

Ask a follow-up question

The n8n workflow is triggered at **`http://localhost:5678/webhook/get-linkedin-jobs`**. `N8n.controller.js:330-337`

JWT authentication supports both HttpOnly cookie and Bearer header modes. `authMiddleware.js:14-24`

The global layout wraps all pages in **`QueryClientProviderWrapper`**, **`AuthProvider`**, and **`NotificationProvider`**. `layout.js:33-52`

The feedback label map assigns ordinal values (0–4) to user interaction types for LTR training. `feedback.service.js:21-27`

The **`skillSyncService.js`** processes Jobs, Steps, and Lessons in three sequential passes to populate missing skill vectors. `skillSyncService.js:9-12`

The Next.js middleware protects **`/admin`**, **`/super-admin`**, **`/profile`**, and **`/community`** routes by checking for the **`token`** cookie. `middleware.js:6-53`

The backend registers 18 distinct route namespaces covering auth, content, community, AI, and automation domains. `server.js:126-173`

KARIM2206/guidfy guidfy/app/components/HeroSection.jsx

```
47              transition={{ duration: 0.7 }}
48            >
49              {/* Main Heading */}
50              <h3 className="text-4xl md:text-5xl lg:text-6xl text-gray-800 font-bold leading-snug
51                Learn, Build,{" "}
52                <span className="bg-gradient-to-r from-blue-600 to-cyan-500 bg-clip-text text-trans
53                  Get Seen
54                </span>
55              </h3>
56
57              {/* Subheading */}
58              <p className="text-lg md:text-xl lg:text-2xl text-gray-600 mb-4 font-medium">
```

KARIM2206/guidfy backend/prisma/schema.prisma

```
7   provider = "prisma-client-js"
```

Ask a follow-up question

```
15
16  enum NotificationType {
17    LIKE
```

JS KARIM2206/guidfy backend/server.js

```
16  require('./prismaClient');
17
18  const app  = express();
19  const PORT = process.env.PORT || 8000;
20
21  // =====================
22  // SECURITY — HELMET
```

```
123  // =====================
124  // ROUTES
125  // =====================
126  const authRoute          = require('./routes/authRoute');
127  const roadmapRoute       = require('./routes/admin/roadmapRoute');
```

← KARIM2206/guidfy

SYSTEM PROMPT — GUIDFY ENTERPRISE ACADEMIC DOCUMENTATION (STRICT MODE) You are a Principal Software Architect, Senior Technical Writer, Solution Architect, Software Engineering Researcher, UML Spec...

Show full text

Deep View as codemap

Thought Process

Let me gather comprehensive codebase information in parallel to produce Chapter 2 with full implementation evidence.

I now have all the information needed. Producing Chapter 2 — System Analysis in full.

Ask a follow-up question

2.1 Stakeholders

A stakeholder is any individual, group, or organization that has an interest in, or is affected by, the Guidfy platform. The following stakeholders were identified through analysis of the implemented system, its user roles, and its external integrations.

Table 2.1 — Stakeholder Register

ID	Stakeholder	Type	Interest	Influence
SH-01	Students / Learners	Primary User	Career guidance, structured learning, job matching	High
SH-02	Platform Administrators (ADMIN)	Internal Operator	Content management, community moderation	High
SH-03	Super Administrator (SUPER_ADMIN)	Platform Owner	Platform governance, analytics, AI management	Very High
SH-04	Employers / Recruiters	External Beneficiary	Access to skilled candidates via job postings	Medium
SH-05	Development Team	Internal	System maintenance, feature development	Very High
SH-06	University / Academic Supervisors	External	Project evaluation and academic oversight	Medium
SH-07	n8n Automation Service	External System	Job data pipeline execution	Low
SH-08	SerpAPI	External System	Job listing data provider	Low
SH-09	AI Service (FastAPI)	Internal System	Recommendation and skill extraction	High

2.2 User Roles

Ask a follow-up question

```
STUDENT: 1 → ADMIN: 2 → SUPER_ADMIN: 3
```

`role.middleware.js:145`

2.2.1 STUDENT

The default role assigned to all newly registered users. Students are the primary consumers of the platform's learning, community, and job recommendation features.

Capabilities confirmed from routes and controllers:

- Register and authenticate
- Enroll in Learning Paths (`POST /api/student/enroll`)
- Track lesson and step progress (`PUT /api/student/lessons/progress`)
- Access roadmaps, steps, and lessons
- Submit quiz attempts (`POST /api/quiz/:quizId/submit`)
- Participate in communities (join, leave, post, question, answer, vote)
- Receive AI-powered job recommendations (`GET /api/recommendations`)
- Submit job interaction feedback (`POST /api/feedback`)
- Save and apply to jobs
- Manage personal profile (avatar, cover, bio, title)
- Send and receive direct messages
- Receive real-time notifications via Socket.io
- Generate AI career reports (`POST /api/career-report/generate`)
- Request AI learning path recommendations (`POST /api/super-admin/learning-paths/recommend`) `studentRoute.js:1-14`

2.2.2 ADMIN

Administrators are assigned by the Super Admin and are responsible for content creation and community moderation within their assigned Learning Paths.

Capabilities confirmed from routes:

- All STUDENT capabilities

Ask a follow-up question

- Manage community members (ban, unban, remove) within assigned communities
- Delete posts and questions in managed communities
- Resolve and dismiss community reports
- View community analytics and action logs
- Assign badges to users (`POST /api/users/admin/badges/assign`)
- Send notifications to specific roles (`POST /api/notifications/send-to-role`)

  adminRoute.js:27-59

2.2.3 SUPER_ADMIN

The Super Administrator has unrestricted access to all platform functions and is the sole authority for platform governance.

Capabilities confirmed from routes:

- All ADMIN capabilities
- Create, update, delete, and publish Learning Paths (`POST /api/super-admin/learning-paths`)
- Create, update, delete, and publish Roadmaps (`POST /api/admin/roadmaps`)
- Assign Admins to Learning Paths and Sections (`POST /api/super-admin/learning-paths/:id/admins`)
- Manage all users: add, delete, change roles (`POST /api/auth/change-role`)
- Suspend, activate, and delete Communities
- Assign and remove Community Admins
- View platform-wide analytics (`GET /api/super-admin/analytics`)
- Manage all community reports (`GET /api/super-admin/reports`)
- Control the job scraper (start, stop, view logs) (`POST /api/admin/jobs/scraper/start`)
- Approve and reject imported jobs (`POST /api/admin/jobs/imported/:id/approve`)
- Trigger AI model retraining (`POST /api/super-admin/ai-training/retrain`)
- Broadcast notifications to all users (`POST /api/notifications/broadcast`)

  superAdminRoute.js:32-88

Ask a follow-up question

Table 2.2 — Business Rules

ID	Rule	Implementation Evidence
BR-01	A user's email must be unique across the platform	`User.email @unique` in schema; `findUnique` check in `register`
BR-02	Passwords are hashed with bcrypt (cost factor 12) before storage	`bcrypt.hash(password, 12)` in `auth.controller.js`
BR-03	JWT tokens are stored in HttpOnly cookies with 30-day expiry	`cookieOptions()` in `auth.controller.js`
BR-04	Authentication endpoints are rate-limited to 20 requests per 15 minutes	`authLimiter` in `server.js`
BR-05	A Roadmap must have a unique slug	`Roadmap.slug @unique` in schema
BR-06	A Learning Path title must be unique	`LearningPath.title @unique` in schema
BR-07	A Community name and slug must be unique	`Community.name @unique` , `Community.slug @unique`
BR-08	A user can only enroll in a Learning Path once	`UserLearningPath @@unique([userId, learningPathId])`
BR-09	A user can only vote once per target (question/answer/post)	`Vote @@unique([userId, targetId, targetType])`
BR-10	A user can only apply to a job once	`JobApplication @@unique([jobId, userId])`
BR-11	A user can only save a job once	`UserSavedJob @@unique([userId, jobId])`
BR-12	Steps within a Roadmap must have unique order values	`Step @@unique([roadmapId, order])`
BR-13	Lessons within a Step must have unique order values	`Lesson @@unique([stepId, order])`
BR-14	An Admin can only manage Jobs in Learning Paths they have JOB section permission for	`LearningPathPermission` check in `jobsRoute.js`
BR-15	A suspended Community cannot be administered	`community.status === 'SUSPENDED'` check in `role.middleware.js`

Ask a follow-up question

ID	Rule	Implementation Evidence
BR-18	The n8n job save endpoint requires an internal token	`validateInternalToken` middleware on `POST /api/n8n/save-jobs`
BR-19	A user can only view a project once (unique view tracking)	`UserProjectView @@unique([userId, projectId])`
BR-20	A user can only view a post/question once (unique view tracking)	`View @@unique([userId, targetId, targetType])`
BR-21	Feedback labels are ordinal: dismiss=0, impression=1, click=2, save=3, apply=4	`LABEL_MAP` in `feedback.service.js`
BR-22	Job recommendations require the user to be enrolled in at least one Learning Path	`enrollments.length` check in `recommendation.service.js`
BR-23	A Roadmap can only be assigned to a Learning Path once	`LearningPathRoadmap @@unique([learningPathId, roadmapId])`
BR-24	A user can only have one badge of each type	`UserBadge @@unique([userId, badgeId])`

2.4 Functional Requirements

Functional requirements describe what the system must do. They are organized by domain module.

2.4.1 Authentication & User Management

Table 2.3 — Authentication Functional Requirements

ID	Requirement	Priority	Source
FR-AUTH-01	The system shall allow users to register with email, name, password, and confirm password	Must Have	`POST /api/auth/register`
FR-AUTH-02	The system shall validate that passwords match during registration	Must Have	`auth.controller.js`
FR-AUTH	The system shall reject registration if the email already exists	Must Have	`auth.controller.j`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-AUTH-05	The system shall issue a JWT stored in an HttpOnly cookie upon successful login	Must Have	`auth.controller.js`
FR-AUTH-06	The system shall also return the JWT in the response body for backward compatibility	Should Have	`auth.controller.js`
FR-AUTH-07	The system shall allow users to log out by clearing the HttpOnly cookie	Must Have	`POST /api/auth/logout`
FR-AUTH-08	The system shall provide a `/me` endpoint to retrieve the current authenticated user	Must Have	`GET /api/auth/me`
FR-AUTH-09	The system shall allow SUPER_ADMIN to add users with any role	Must Have	`POST /api/auth/add-user`
FR-AUTH-10	The system shall allow SUPER_ADMIN to change any user's role	Must Have	`POST /api/auth/change-role`
FR-AUTH-11	The system shall allow SUPER_ADMIN to delete users	Must Have	`DELETE /api/auth/delete-user`
FR-AUTH-12	The system shall provide paginated user listing for administrators	Should Have	`GET /api/auth/all-users`
FR-AUTH-13	The frontend middleware shall redirect unauthenticated users from protected routes to `/login`	Must Have	`guidfy/middleware.js`

2.4.2 Learning Content Management

Table 2.4 — Content Management Functional Requirements

ID	Requirement	Priority	Source
FR-CONT-01	The system shall allow SUPER_ADMIN to create, update, delete, and publish Learning Paths	Must Have	`learningPathRoute.js`
FR-	The system shall allow SUPER_ADMIN to create, update, delete,	Must	`roadmapRoute.j`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-CONT-04	The system shall support three Lesson types: VIDEO, ARTICLE, and QUIZ	Must Have	`LessonType` enum in schema
FR-CONT-05	The system shall allow ADMIN to add Video Lessons with external URL or file upload	Must Have	`lessonRoute.js`
FR-CONT-06	The system shall allow ADMIN to add Article Lessons with rich text content	Must Have	`lessonRoute.js`
FR-CONT-07	The system shall allow ADMIN to create Quiz Lessons with questions and options	Must Have	`quizRoute.js`
FR-CONT-08	The system shall track student progress at the Lesson, Step, and Roadmap levels	Must Have	`studentRoute.js`
FR-CONT-09	The system shall allow students to enroll in Learning Paths	Must Have	`POST /api/student/enroll`
FR-CONT-10	The system shall allow SUPER_ADMIN to assign Admins to Learning Path sections (ROADMAP, JOB, PROJECT)	Must Have	`learningPathRoute.js`
FR-CONT-11	The system shall support Roadmap levels: BEGINNER, INTERMEDIATE, ADVANCED	Should Have	`LevelRoadmap` enum
FR-CONT-12	The system shall support Roadmap statuses: DRAFT, ASSIGNED, IN_PROGRESS, REVIEW, NEEDS_REVISION, PUBLISHED	Should Have	`RoadmapStatus` enum

2.4.3 Community Platform

Table 2.5 — Community Functional Requirements

ID	Requirement	Priority	Source
FR-COMM-01	The system shall allow ADMIN/SUPER_ADMIN to create, update, and delete Communities	Must Have	`communityRoute.js`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-COMM-03	The system shall allow community members to create Posts with title, body, image, and tags	Must Have	`communityRoute.js`
FR-COMM-04	The system shall allow community members to create Questions with title, body, and tags	Must Have	`communityRoute.js`
FR-COMM-05	The system shall allow community members to post Answers to Questions	Must Have	`communityRoute.js`
FR-COMM-06	The system shall allow users to upvote and downvote Questions, Answers, and Posts	Must Have	`POST /api/communities/vote`
FR-COMM-07	The system shall support nested comment replies on Posts	Must Have	`PostComment.parentId` in schema
FR-COMM-08	The system shall track unique view counts for Posts and Questions	Should Have	`View` model in schema
FR-COMM-09	The system shall allow users to report Posts, Questions, Communities, and Users	Must Have	`POST /api/reports`
FR-COMM-10	The system shall allow ADMIN to ban, unban, and remove community members	Must Have	`adminRoute.js`
FR-COMM-11	The system shall allow ADMIN to delete posts and questions in managed communities	Must Have	`adminRoute.js`
FR-COMM-12	The system shall allow ADMIN to resolve and dismiss community reports	Must Have	`adminRoute.js`
FR-COMM-13	The system shall allow SUPER_ADMIN to suspend and activate Communities	Must Have	`superAdminRoute.js`
FR-COMM-14	The system shall provide a global community feed	Should Have	`GET /api/communities/feed`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-COMM-16	The system shall maintain an admin action log for all moderation actions	Should Have	`AdminActionLog` model

2.4.4 Job Recommendation System

Table 2.6 — Job Recommendation Functional Requirements

ID	Requirement	Priority	Source
FR-JOB-01	The system shall provide AI-powered job recommendations for enrolled students	Must Have	`GET /api/recommendations`
FR-JOB-02	The system shall derive user skills from completed Steps and Lessons	Must Have	`recommendation.service.js`
FR-JOB-03	The system shall send user profile and job data to the FastAPI AI service for ranking	Must Have	`recommendation.service.js`
FR-JOB-04	The system shall record user feedback on job recommendations (apply, save, click, dismiss, impression)	Must Have	`POST /api/feedback`
FR-JOB-05	The system shall forward feedback to the AI service for model improvement	Must Have	`feedback.service.js`
FR-JOB-06	The system shall allow SUPER_ADMIN to trigger AI model retraining	Must Have	`POST /api/super-admin/ai-training/retrain`
FR-JOB-07	The system shall allow SUPER_ADMIN to trigger automated job scraping via n8n	Must Have	`POST /api/admin/jobs/scraper/start`
FR-JOB-08	The system shall allow SUPER_ADMIN to approve or reject scraped jobs before publishing	Must Have	`jobsRoute.js`
FR-JOB-09	The system shall allow students to save jobs	Must Have	`POST /api/admin/jobs/:jobId/save`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-JOB-11	The system shall provide a career report generation feature	Should Have	`POST /api/career-report/generate`

2.4.5 Real-time Communication

Table 2.7 — Real-time Communication Functional Requirements

ID	Requirement	Priority	Source
FR-RT-01	The system shall deliver real-time notifications to authenticated users via Socket.io	Must Have	`server.js` Socket.io setup
FR-RT-02	Each authenticated user shall be assigned a private Socket.io room (`user_{userId}`)	Must Have	`socket.on('register', ...)` in `server.js`
FR-RT-03	The system shall allow users to send and receive direct messages	Must Have	`POST /api/chat/messages`
FR-RT-04	The system shall provide a conversation history endpoint	Must Have	`GET /api/chat/messages/:userId`
FR-RT-05	The system shall provide an inbox endpoint listing all conversations	Must Have	`GET /api/chat/inbox`
FR-RT-06	The system shall allow ADMIN/SUPER_ADMIN to send notifications to specific roles	Must Have	`POST /api/notifications/send-to-role`
FR-RT-07	The system shall allow SUPER_ADMIN to broadcast notifications to all users	Must Have	`POST /api/notifications/broadcast`
FR-RT-08	The system shall allow users to mark notifications as read individually or all at once	Must Have	`notificationRoutes.js`

2.4.6 User Profile & Portfolio

Table 2.8 — Profile & Portfolio Functional Requirements

ID	Requirement	Priority	Source
FR-PROF-01	The system shall allow users to update their profile (name, bio, title, experience years)	Must Have	`PUT /api/users/profile`

Ask a follow-up question

ID	Requirement	Priority	Source
FR-PROF-03	The system shall allow users to upload a cover image	Must Have	`POST /api/users/cover`
FR-PROF-04	The system shall display user statistics (reputation, followers, following)	Should Have	`UserStats` model
FR-PROF-05	The system shall display user badges	Should Have	`UserBadge` model
FR-PROF-06	The system shall display user track progress	Should Have	`TrackProgress` model
FR-PROF-07	The system shall allow ADMIN to create Projects associated with Learning Paths	Must Have	`POST /api/admin/projects`
FR-PROF-08	The system shall track unique project views	Should Have	`UserProjectView` model

2.4.7 AI Learning Path Recommendation

Table 2.9 — AI Learning Path Recommendation Functional Requirements

ID	Requirement	Priority	Source
FR-AI-01	The system shall present a multi-step questionnaire to collect user experience, skills, interests, and daily commitment	Must Have	`user-journey.spec.js`
FR-AI-02	The system shall submit questionnaire responses to the AI service for Learning Path recommendation	Must Have	`POST /api/super-admin/learning-paths/recommend`
FR-AI-03	The system shall store the recommendation result including confidence score and top paths	Must Have	`LearningPathRecommendation` model
FR-AI-04	The system shall display the recommended Learning Path to the user	Must Have	`user-journey.spec.js`
FR-AI-05	The system shall allow users to retrieve their previous recommendations	Must Have	`GET /api/super-admin/learning-paths/my-recommendations`

Ask a follow-up question

2.5.1 Security Requirements

Table 2.10 — Security Non-Functional Requirements

ID	Requirement	Implementation
NFR-SEC-01	All API responses must include security headers	Helmet middleware in `server.js`
NFR-SEC-02	CORS must be restricted to configured allowed origins	`corsOptions` in `server.js`
NFR-SEC-03	Authentication tokens must be stored in HttpOnly cookies to prevent XSS access	`cookieOptions()` in `auth.controller.js`
NFR-SEC-04	Passwords must be hashed using bcrypt with a minimum cost factor of 12	`bcrypt.hash(password, 12)`
NFR-SEC-05	Authentication endpoints must be rate-limited (20 requests / 15 minutes)	`authLimiter` in `server.js`
NFR-SEC-06	Internal service-to-service communication must use a shared secret token	`validateInternalToken` middleware
NFR-SEC-07	Role-based access must be enforced at the API layer for every protected endpoint	`requireAdmin` , `requireSuperAdmin` , `allowedTo` middleware
NFR-SEC-08	Frontend route protection must redirect unauthenticated users	`guidfy/middleware.js`
NFR-SEC-09	File uploads must be handled by Multer with type and size restrictions	Upload middleware files
NFR-SEC-10	Request body size must be limited to 10MB	`express.json({ limit: '10mb' })` in `server.js`

2.5.2 Performance Requirements

Table 2.11 — Performance Non-Functional Requirements

ID	Requirement	Implementation
NFR-PERF-01	The AI recommendation service must respond within 120 seconds (timeout)	`timeout: 120_000` in `recommendation.service.js`
NFR-PERF-02	The feedback service must respond within 90 seconds	`timeout: 90_000` in

Ask a follow-up question

ID	Requirement	Implementation
NFR-PERF-04	The recommendation engine must scan up to 150 jobs per request	`take: 150` in `recommendation.service.js`
NFR-PERF-05	TanStack Query must cache API responses with configurable stale time	`staleTime: Infinity` for auth in `AuthProvider.jsx`
NFR-PERF-06	Dashboard statistics must be computed in parallel using `Promise.all`	`Promise.all` in `getAllDashboardStats`
NFR-PERF-07	User pagination must be limited to a maximum of 100 records per page	`Math.min(100, ...)` in `getAllUsers`

2.5.3 Scalability Requirements

ID	Requirement	Notes
NFR-SCALE-01	The AI service must be independently deployable and scalable	Separate FastAPI process on port 4000
NFR-SCALE-02	The n8n automation layer must be independently deployable	Separate n8n process on port 5678
NFR-SCALE-03	The Prisma singleton pattern must prevent connection pool exhaustion	`prismaClient.js` singleton
NFR-SCALE-04	Socket.io must support room-based broadcasting for targeted delivery	`user_{userId}` room pattern
NFR-SCALE-05	The skill sync pipeline must process entities in batches	`batchExtractSkills` in `skillSyncService.js`

2.5.4 Reliability Requirements

ID	Requirement	Implementation
NFR-REL-01	The feedback notification to the AI service must be fire-and-forget (non-blocking)	`.catch(err => ...)` in `feedback.service.js`
NFR-REL-02	The system must handle AI service unavailability gracefully	`ECONNREFUSED` check in `recommendationRoute.js`
NFR-REL-03	The global error middleware must catch all unhandled errors	`notFound + errorHandlerMiddleware` in `server.js`
NFR-	The Playwright E2E tests must use randomized emails	`Date.now() + Math.random()` in

Ask a follow-up question

ID	Requirement	Implementation
NFR-USE-01	The platform must be responsive across desktop and mobile viewports	TailwindCSS responsive utilities
NFR-USE-02	Toast notifications must provide immediate feedback for user actions	React Toastify in `layout.js`
NFR-USE-03	The authentication state must persist across page refreshes	HttpOnly cookie + `GET /api/auth/me` on mount
NFR-USE-04	The AI questionnaire must guide users through a multi-step flow	4-step questionnaire confirmed in `user-journey.spec.js`

2.6 User Stories

User stories are written in the standard format: **As a [role], I want to [action], so that [benefit].**

Table 2.12 — User Stories

ID	Role	Story	Priority
US-01	Student	As a student, I want to register with my email and password so that I can access the platform	Must Have
US-02	Student	As a student, I want to log in and have my session persist so that I don't need to re-authenticate on every visit	Must Have
US-03	Student	As a student, I want to take an AI questionnaire so that I receive a personalized Learning Path recommendation	Must Have
US-04	Student	As a student, I want to enroll in a Learning Path so that I can follow a structured career track	Must Have
US-05	Student	As a student, I want to track my progress through Lessons and Steps so that I know how far I've advanced	Must Have
US-06	Student	As a student, I want to receive AI-powered job recommendations based on my acquired skills so that I can find relevant employment opportunities	Must Have
US-07	Student	As a student, I want to save jobs I'm interested in so that I can review them later	Should Have

Ask a follow-up question

ID	Role	Story	Priority
US-10	Student	As a student, I want to receive real-time notifications so that I stay informed about platform activity	Should Have
US-11	Student	As a student, I want to send direct messages to other users so that I can communicate privately	Should Have
US-12	Student	As a student, I want to update my profile with a bio, title, and avatar so that I present myself professionally	Should Have
US-13	Student	As a student, I want to take quizzes at the end of lessons so that I can assess my understanding	Should Have
US-14	Student	As a student, I want to generate an AI career report so that I understand my career readiness	Could Have
US-15	Admin	As an admin, I want to create Steps and Lessons within assigned Roadmaps so that I can build learning content	Must Have
US-16	Admin	As an admin, I want to manage community members (ban/unban) so that I can maintain a healthy community	Must Have
US-17	Admin	As an admin, I want to resolve community reports so that I can address inappropriate content	Must Have
US-18	Admin	As an admin, I want to view community analytics so that I can understand engagement	Should Have
US-19	Admin	As an admin, I want to create and manage job listings so that students have relevant opportunities	Must Have
US-20	Super Admin	As a super admin, I want to create and publish Learning Paths so that students have structured career tracks	Must Have
US-21	Super Admin	As a super admin, I want to assign admins to Learning Path sections so that content management is delegated	Must Have
US-22	Super Admin	As a super admin, I want to manage all users (add, delete, change roles) so that I control platform access	Must Have
US-23	Super Admin	As a super admin, I want to trigger the job scraping pipeline so that the job database stays current	Must Have
US-24	Super Admin	As a super admin, I want to approve or reject scraped jobs so that only quality listings are published	Must Have
US-	Super	As a super admin, I want to trigger AI model retraining so that	Must

Ask a follow-up question

ID	Role	Story	Priority
US-27	Super Admin	As a super admin, I want to suspend communities that violate platform rules so that I maintain platform integrity	Must Have
US-28	Super Admin	As a super admin, I want to broadcast notifications to all users so that I can communicate platform-wide announcements	Should Have

---

2.7 Acceptance Criteria

Selected acceptance criteria for the highest-priority user stories, derived from the Playwright test suite and route implementations.

US-01 / US-02 — Registration and Login

Acceptance Criteria (confirmed by `login.spec.js` and `user-journey.spec.js`):

- AC-01.1: Given a user provides a valid unique email, name, password, and matching confirm password, when they submit the registration form, then the system creates the account and returns HTTP 201.
- AC-01.2: Given a user provides an email that already exists, when they submit the registration form, then the system returns HTTP 400 with error "Email already exists".
- AC-01.3: Given a user provides mismatched passwords, when they submit the registration form, then the system returns HTTP 400 with error "Passwords do not match".
- AC-02.1: Given a registered user provides correct credentials, when they submit the login form, then the system sets an HttpOnly `token` cookie and returns user data.
- AC-02.2: Given a logged-in user, when they navigate to a protected route, then the Next.js middleware allows access.
- AC-02.3: Given an unauthenticated user, when they navigate to `/admin`, `/super-admin`, `/profile`, or `/community`, then they are redirected to `/login?from=<pathname>`.
  `login.spec.js:1-40`

US-03 — AI Learning Path Questionnaire

Acceptance Criteria (confirmed by `user-journey.spec.js`):

Ask a follow-up question

- AC-03.2: The questionnaire must present four steps: (1) coding experience, (2) known technologies, (3) career interest, (4) daily hours commitment.
- AC-03.3: Given the student completes all four steps and submits, when the AI service responds, then the heading "Your Recommended Path" must be visible within 30 seconds.
- AC-03.4: Given the recommendation is displayed, when the student clicks "View My Path", then the main page shows an "AI Recommended Path" banner. `user-journey.spec.js:55-88`

US-15 — Admin Content Creation

Acceptance Criteria (confirmed by `admin-dashboard.spec.js`):

- AC-15.1: Given an admin is logged in and navigates to a Roadmap, when they click "Add Step", then a modal appears with title and description fields.
- AC-15.2: Given the admin fills in the step details and clicks "Save Step", then the new step appears in the roadmap view.
- AC-15.3: Given the admin expands a step and clicks "Add Lesson", then a lesson creation form appears.
- AC-15.4: Given the admin selects "External" video type and provides a URL, when they click "Save Lesson", then the lesson appears under the step. `admin-dashboard.spec.js:48-90`

US-25 — AI Model Retraining

Acceptance Criteria (confirmed by `super-admin-dashboard.spec.js`):

- AC-25.1: Given a super admin navigates to the "AI Training" section, when they click "Refresh", then the training status is updated.
- AC-25.2: The AI Training section must be accessible only to SUPER_ADMIN role. `super-admin-dashboard.spec.js:147-155`

2.8 System Constraints

ID	Constraint	Type	Description
SC-01	MySQL database	Technical	The system is bound to MySQL as the relational database provider via Prisma

Ask a follow-up question

ID	Constraint	Type	Description
SC-04	n8n instance	Operational	Job scraping requires a running n8n instance on port 5678
SC-05	Single-server deployment	Operational	The current architecture assumes all services run on a single host (localhost references)
SC-06	No Docker containerization	Operational	No Dockerfile or docker-compose.yml was found in the indexed codebase
SC-07	English-only interface	Functional	The platform operates exclusively in English; no localization is implemented
SC-08	No mobile native app	Functional	The platform is web-only; no React Native or mobile application exists
SC-09	No payment processing	Functional	Stripe is listed as a reference but not implemented in the indexed codebase
SC-10	WebKit browser limitation	Testing	Playwright tests skip WebKit due to ITP cookie blocking on localhost cross-port requests

2.9 Assumptions

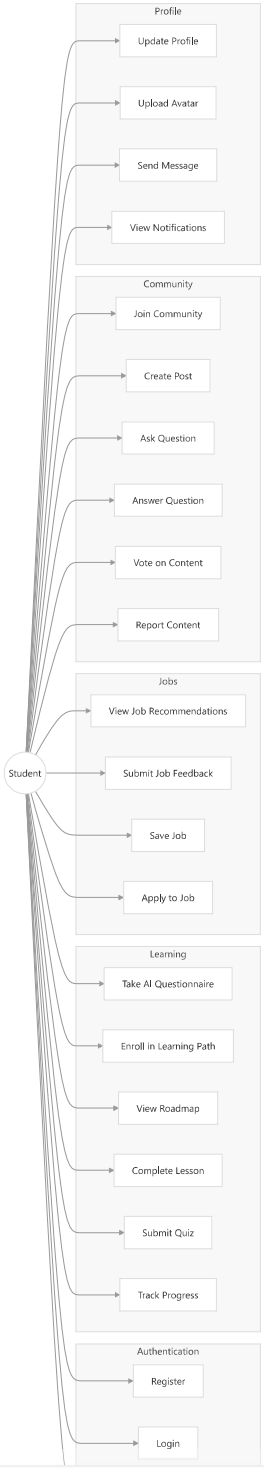
ID	Assumption
AS-01	All three services (Next.js, Express, FastAPI) are running simultaneously during normal operation
AS-02	The MySQL database is accessible via the `DATABASE_URL` environment variable
AS-03	The n8n instance is configured with the LinkedIn job scraping workflow
AS-04	The `INTERNAL_API_TOKEN` environment variable is shared between the Express backend and the n8n instance
AS-05	The AI service has been pre-trained with an initial LightGBM model before the first recommendation request
AS-06	Users have access to a modern web browser supporting WebSocket connections
AS-	Email delivery via Nodemailer requires a valid SMTP configuration in environment variables

Ask a follow-up question

2.10 Use Cases

2.10.1 Use Case Diagram — Student Domain

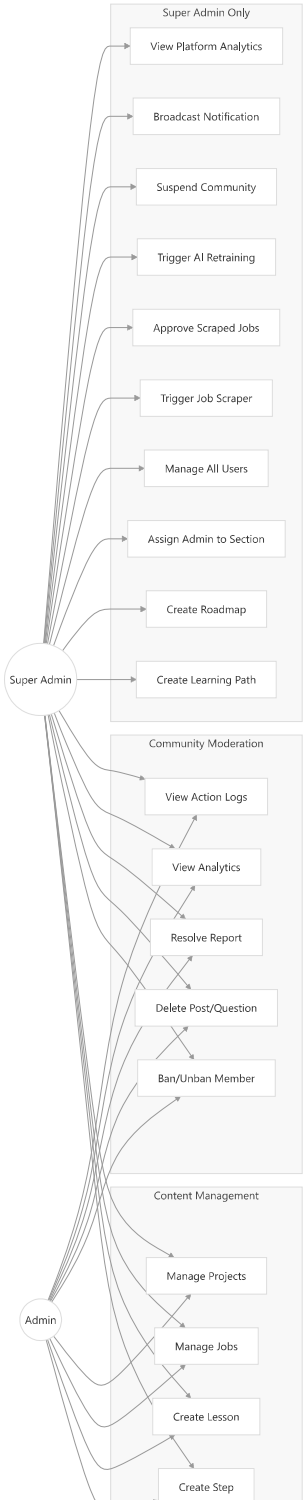
Figure 2.1 — Student Use Case Diagram



Ask a follow-up question

2.10.2 Use Case Diagram — Admin Domain

Figure 2.2 — Admin Use Case Diagram



Ask a follow-up question

2.10.3 Use Case Diagram — System-Level

Figure 2.3 — System-Level Use Case Diagram

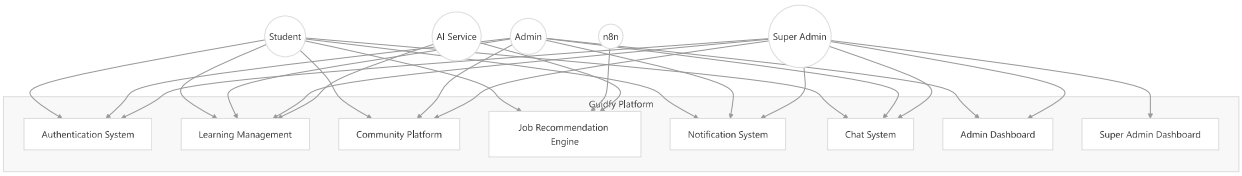


Figure 2.3: System-level use case diagram showing all actors and their interactions with the major platform subsystems.

2.11 Use Case Descriptions

UC-01: Student Registration

Field	Detail
Use Case ID	UC-01
Name	Student Registration
Actor	Unauthenticated User
Precondition	User is not registered on the platform
Trigger	User navigates to `/register` and submits the registration form
Main Flow	1. User provides name, email, password, confirm password. 2. System validates all fields are present. 3. System validates passwords match. 4. System checks email uniqueness. 5. System hashes password with bcrypt (cost 12). 6. System creates User record with default role STUDENT. 7. System returns HTTP 201 success.
Alternative Flow	4a. Email exists → return HTTP 400 "Email already exists". 3a. Passwords mismatch → return HTTP 400 "Passwords do not match".

Ask a follow-up question

Field	Detail
Implementation	`POST /api/auth/register` → `auth.controller.js:register`

UC-02: AI Learning Path Recommendation

Field	Detail
Use Case ID	UC-02
Name	AI Learning Path Recommendation
Actor	Authenticated Student
Precondition	Student is logged in and has not yet received a recommendation (or wants to redo)
Trigger	Student clicks "Find My Path" on the Learning Paths page
Main Flow	1. System displays 4-step questionnaire modal. 2. Step 1: Student selects experience level (Beginner/Intermediate/Advanced). 3. Step 2: Student selects known technologies. 4. Step 3: Student selects career interest (Frontend/Backend/AI-ML/etc.). 5. Step 4: Student selects daily hours commitment. 6. Student submits. 7. Backend sends questionnaire data to AI service. 8. AI service returns recommended path with confidence score. 9. System stores `LearningPathRecommendation` record. 10. System displays recommendation to student.
Alternative Flow	7a. AI service unavailable → return error message to user.
Postcondition	`LearningPathRecommendation` record created; student sees recommended path
Impl	`POST /api/super-admin/learning-paths/recommend` →

Ask a follow-up question

UC-03: Job Recommendation

Field	Detail
Use Case ID	UC-03
Name	AI Job Recommendation
Actor	Authenticated Student
Precondition	Student is enrolled in at least one Learning Path
Trigger	Student navigates to the job recommendations section
Main Flow	1. Backend fetches student's Learning Path enrollments. 2. Backend derives user skills from completed Steps and Lessons. 3. Backend fetches up to 150 active jobs from database. 4. Backend sends user profile + jobs to FastAPI `/recommend` endpoint. 5. AI service ranks jobs using LightGBM LTR model. 6. Backend enriches results with skill gap analysis and explanations. 7. System returns ranked job list with match scores (0–100).
Alternative Flow	1a. No enrollment → return `{ reason: 'no_enrollment' }`. 3a. No jobs → return `{ reason: 'no_jobs' }`. 4a. AI service connection refused → return HTTP 503.
Postcondition	Student receives personalized ranked job list
Implementation	`GET /api/recommendations → recommendation.service.js:getRecommendations`

UC-04: Community Moderation (Admin)

Field	Detail
Use Case ID	UC-04

Ask a follow-up question

Field	Detail
Precondition	Admin is assigned to manage the community; community is ACTIVE
Trigger	Admin navigates to community management dashboard
Main Flow	1. Admin views community members, posts, questions, and reports. 2. Admin selects a reported item. 3. Admin reviews the report. 4. Admin resolves (removes content) or dismisses the report. 5. System logs the action in `AdminActionLog`. 6. System updates report status to RESOLVED or DISMISSED.
Alternative Flow	2a. Admin bans a member → `BannedMember` record created with reason and timestamp.
Postcondition	Report status updated; action logged in `AdminActionLog`
Implementation	`PUT /api/admin/reports/:reportId/resolve` → `admin.controller.js`

UC-05: Job Scraping Pipeline

Field	Detail
Use Case ID	UC-05
Name	Automated Job Scraping
Actor	Super Admin, n8n (System)
Precondition	n8n instance is running; SerpAPI key is configured
Trigger	Super Admin clicks "Start Scraper" or scheduled n8n trigger fires

Ask a follow-up question

Field	Detail
Alternative Flow	4a. Invalid internal token → HTTP 401 rejected.
Post condition	New Job records exist in database pending approval
Implementation	`N8n.controller.js` , `n8nRoutes.js` , `jobsRoute.js`

2.12 Use Case Matrix

Table 2.13 — Use Case to Role Matrix

Use Case	Student	Admin	Super Admin	n8n	AI Service
Register / Login / Logout	✓	✓	✓	—	—
Take AI Questionnaire	✓	✓	✓	—	—
Enroll in Learning Path	✓	—	—	—	—
Track Lesson Progress	✓	—	—	—	—
Submit Quiz	✓	—	—	—	—
View Job Recommendations	✓	—	—	—	—
Submit Job Feedback	✓	—	—	—	—
Join/Leave Community	✓	✓	✓	—	—
Create Post/Question	✓	✓	✓	—	—
Vote on Content	✓	✓	✓	—	—

Ask a follow-up question

Use Case	Student	Admin	Super Admin	n8n	AI Service
Ban/Unban Member	—	✓	✓	—	—
Resolve Report	—	✓	✓	—	—
Create Learning Path	—	—	✓	—	—
Create Roadmap	—	—	✓	—	—
Assign Admin to Section	—	—	✓	—	—
Manage All Users	—	—	✓	—	—
Trigger Job Scraper	—	—	✓	—	—
Approve Scraped Jobs	—	—	✓	—	—
Save Jobs to DB	—	—	—	✓	—
Trigger AI Retraining	—	—	✓	—	—
Suspend Community	—	—	✓	—	—
Broadcast Notification	—	—	✓	—	—
Rank Jobs (LTR)	—	—	—	—	✓
Extract Skills (NLP)	—	—	—	—	✓

2.13 Requirement Traceability Matrix

Table 2.14 — Requirement Traceability Matrix (Selected)

Requirement ID	User Story	Use Case	Backend Route	Controller	Database Model
FR-AUTH-01	US-01	UC-01	`POST /api/auth/register`	`auth.controller.js:register`	`User`
FR-AUTH-04	US-02	—	`POST /api/auth/login`	`auth.controller.js:login`	`User`
FR-	US-02	—	Next.js middleware	`guidfy/middlewar`	—

Ask a follow-up question

Requirement ID	User Story	Use Case	Backend Route	Controller	Database Model
FR-CONT-01	US-20	—	`POST /api/super-admin/learning-paths`	`learningPath.controller.js`	`LearningPath`
FR-CONT-08	US-05	—	`PUT /api/student/lessons/progress`	`student.controller.js`	`UserLessonProgress`
FR-JOB-01	US-06	UC-03	`GET /api/recommendations`	`recommendation.service.js`	`Job`, `UserStepProgress`
FR-JOB-04	US-08	—	`POST /api/feedback`	`feedback.controller.js`	`JobRecommendationFeedback`
FR-JOB-07	US-23	UC-05	`POST /api/admin/jobs/scraper/start`	`N8n.controller.js`	`Job`
FR-COMM-09	US-09	—	`POST /api/reports`	`report.controller.js`	`CommunityReport`
FR-COMM-10	US-16	UC-04	`POST /api/admin/community/:slug/members/:userId/ban`	`admin.controller.js`	`BannedMember`
FR-RT-01	US-10	—	Socket.io `register` event	`server.js`	`Notification`
FR-RT-03	US-11	—	`POST /api/chat/messages`	`chat.controller.js`	`ChatMessage`
FR-PROF-02	US-12	—	`POST /api/users/avatar`	`userProfile.controller.js`	`User.avatar`
FR-JOB-06	US-25	—	`POST /api/super-admin/ai-training/retrain`	`aiTraining.controller.js`	`JobRecommendationFeedback`

2.14 Risk Analysis

Table 2.15 — Risk Register

ID	Risk	Probability	Impact	Mitigation

Ask a follow-up question

ID	Risk	Probability	Impact	Mitigation
R-02	n8n scraping pipeline failure causes stale job data	Medium	Medium	Manual job creation available; scraper status monitoring endpoint
R-03	LightGBM model cold-start with insufficient feedback data	High	Medium	Initial model pre-trained; system functions without retraining
R-04	JWT token theft via XSS	Low	High	HttpOnly cookie prevents JavaScript access to token
R-05	Brute-force authentication attacks	Medium	High	Rate limiter: 20 requests / 15 minutes on auth endpoints
R-06	Database connection pool exhaustion under load	Low	High	Prisma singleton pattern prevents multiple client instances
R-07	Cross-origin request forgery (CSRF)	Low	Medium	SameSite cookie policy; strict CORS origin whitelist
R-08	Unauthorized access to internal n8n endpoint	Low	High	`validateInternalToken` middleware on `POST /api/n8n/save-jobs`
R-09	Large file uploads causing server memory issues	Low	Medium	Multer with size limits; 10MB body limit in Express
R-10	WebKit browser incompatibility with HttpOnly cookies on localhost	High (dev only)	Low	Playwright tests skip WebKit; production uses HTTPS

2.15 Chapter 2 — Diagrams

Figure 2.4 — Authentication Flow Sequence Diagram

Figure 2.4: Authentication sequence diagram showing the complete login flow from form submission through JWT cookie issuance to subsequent authenticated request handling.

Ask a follow-up question

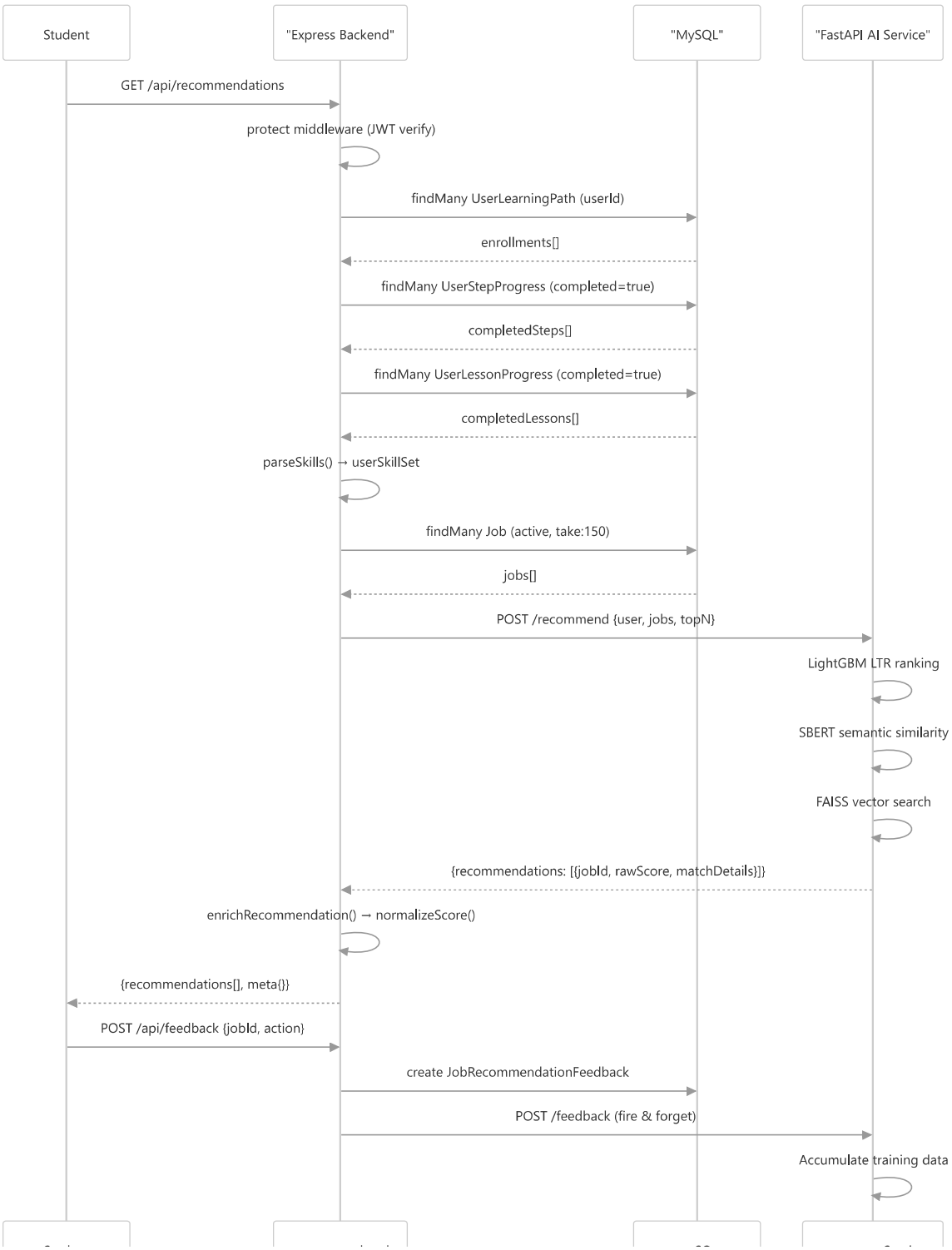


Figure 2.5: Complete job recommendation sequence diagram showing the data flow from student request through skill extraction, AI ranking, result enrichment, and feedback recording.

Ask a follow-up question

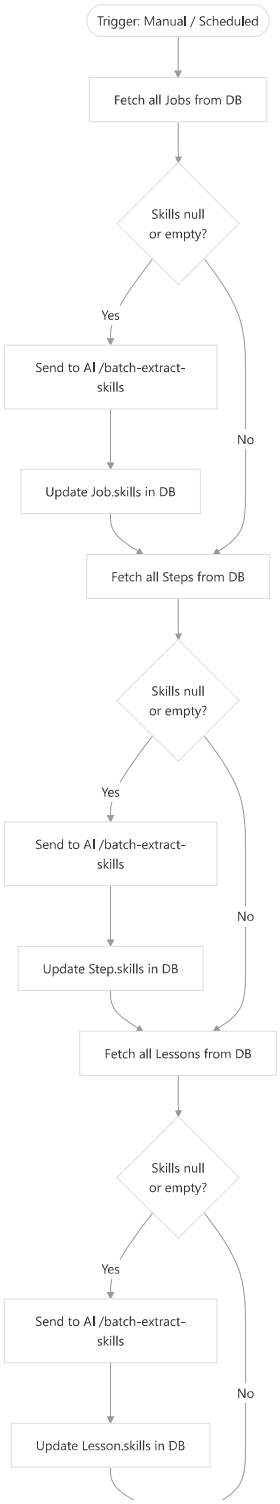


*Figure 2.6: Skill synchronization pipeline activity diagram showing the three-pass process that ensures all

Ask a follow-up question

```
  STUDENT
  SUPER_ADMIN
  ADMIN
}

enum NotificationType {
  LIKE
```

JS KARIM2206/guidfy backend/middleware/role.middleware.js

```
// ═══════════════════════════════════════════
// Role hierarchy check (minimum role)
// ═══════════════════════════════════════════
const roleHierarchy = { STUDENT: 1, ADMIN: 2, SUPER_ADMIN: 3 };

const requireMinRole = (minRole) => {
  return async (req, res, next) => {
```

JS KARIM2206/guidfy backend/routes/student/studentRoute.js

```
const router=require('express').Router()
const { getStudentRoadmaps, getStudentRoadmapStepsById, getStudentLessonsByStepId, updateLessonPr
const {protect}=require('../../middleware/authMiddleware')
const checkUser=require('../../middleware/checkUser')


router.get('/roadmaps/title/:title',protect,checkUser,getStudentRoadmaps)
router.get('/steps/roadmap/:roadmapId',protect,checkUser,getStudentRoadmapStepsById)
router.get('/lessons/step/:stepId',protect,checkUser,getStudentLessonsByStepId)
 router.get('/steps/:stepId/progress',protect,checkUser,getStepProgress)
 router.get('/roadmaps/:roadmapId/progress',protect,checkUser,getRoadmapProgress)
 router.put('/lessons/progress',protect,checkUser,updateLessonProgress)
 router.post('/enroll',protect,checkUser,enrollStudentInLearningPath)
 router.get('/user-content',protect,checkUser,getUserContent)

module.exports=router
```

Ask a follow-up question